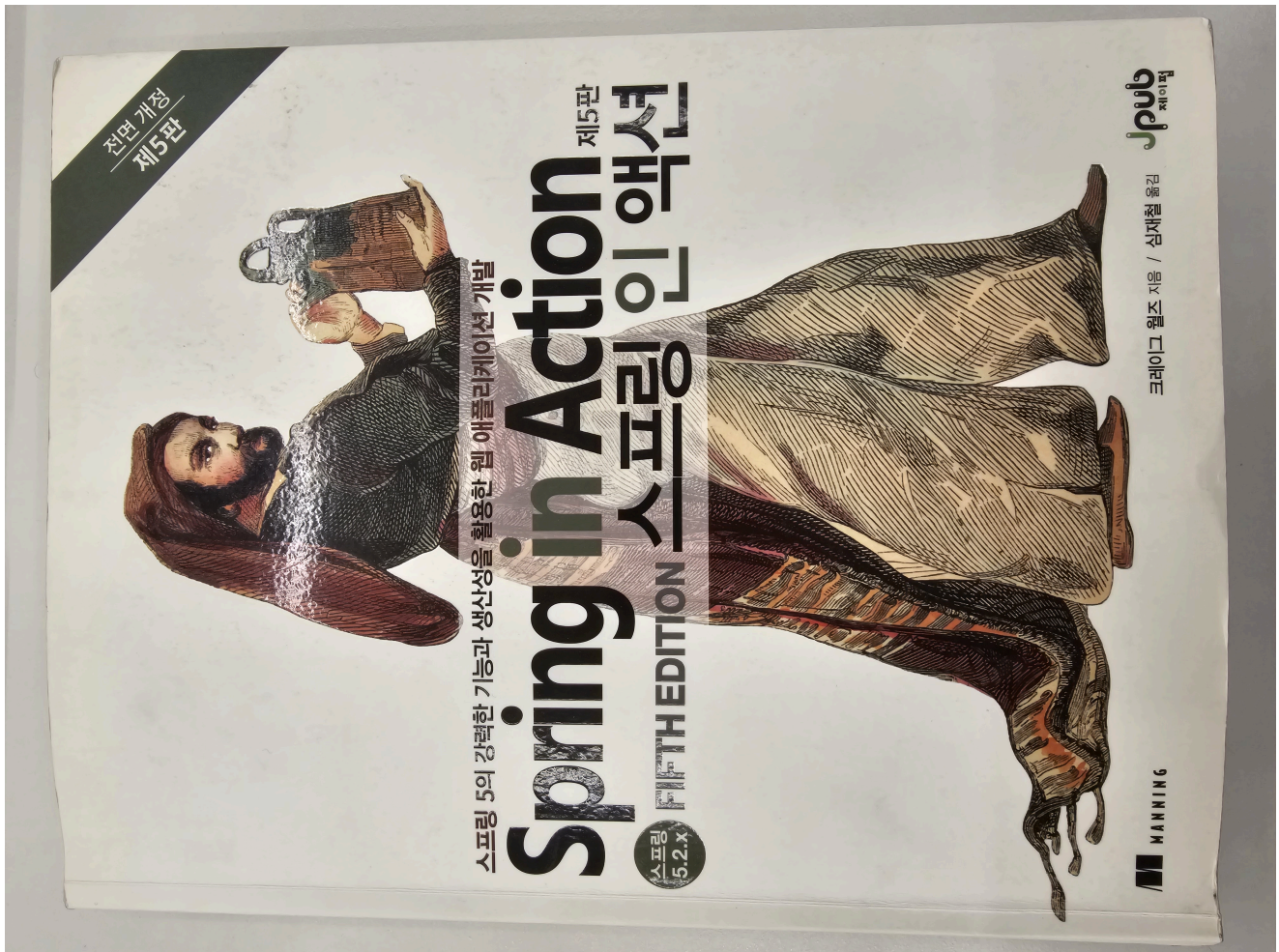


Spring boot - 1

■ Spring in Action 5판 기반 스프링 부트 공부 필기

참고

- 책: *Spring in Action* 제5판 / 크레이그 윌즈 지음, 심재철 옮김
- 코드 및 예시는 책 내용을 기반으로 함
- 나의 학습 환경: Java 17, IntelliJ
- PDF 다운로드: [<https://plusb3b.kr/pdf-download-page/index.html>]
- 책의 공식 GitHub: <https://github.com/Jpub/SpringInAction5>



1.1 스프링이란?

Spring ApplicationContext = 스프링이 제공하는 컨테이너
→ 애플리케이션 컴포넌트를 생성하고 관리함

- **Bean 상호 연결**은 의존성 주입(DI, Dependency Injection) 기반

```
@Configuration
public class ServiceConfiguration {

    @Bean
    public InventoryService inventoryService() {
        return new InventoryService();
    }

    @Bean
    public ProductService productService() {
        return new ProductService(inventoryService());
    }
}
```

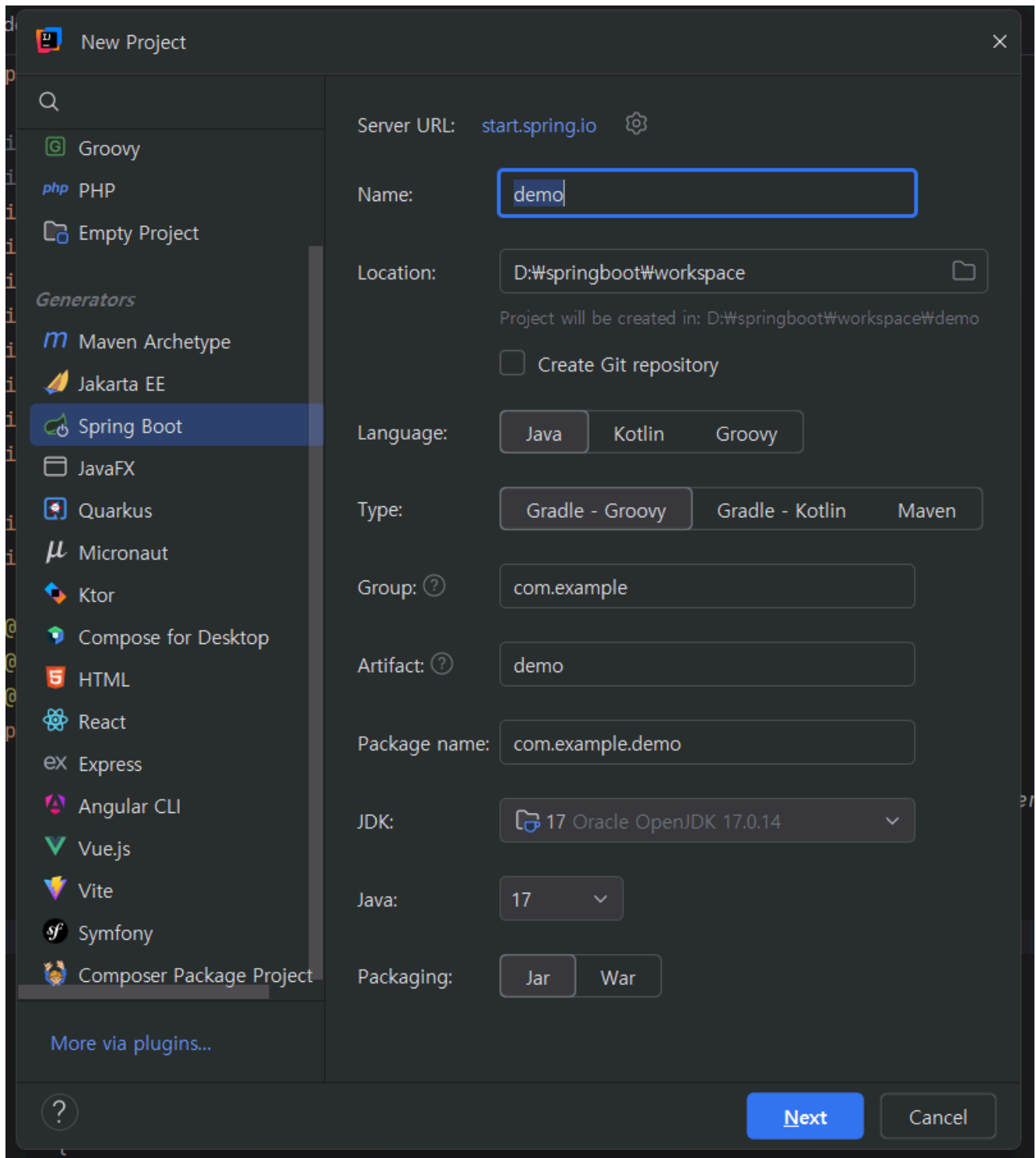
- @Configuration : 구성 클래스
- @Bean : 해당 메서드 리턴 객체를 Bean으로 등록

1.2 스프링 애플리케이션 초기 설정하기

Spring Initializr 사용 방법

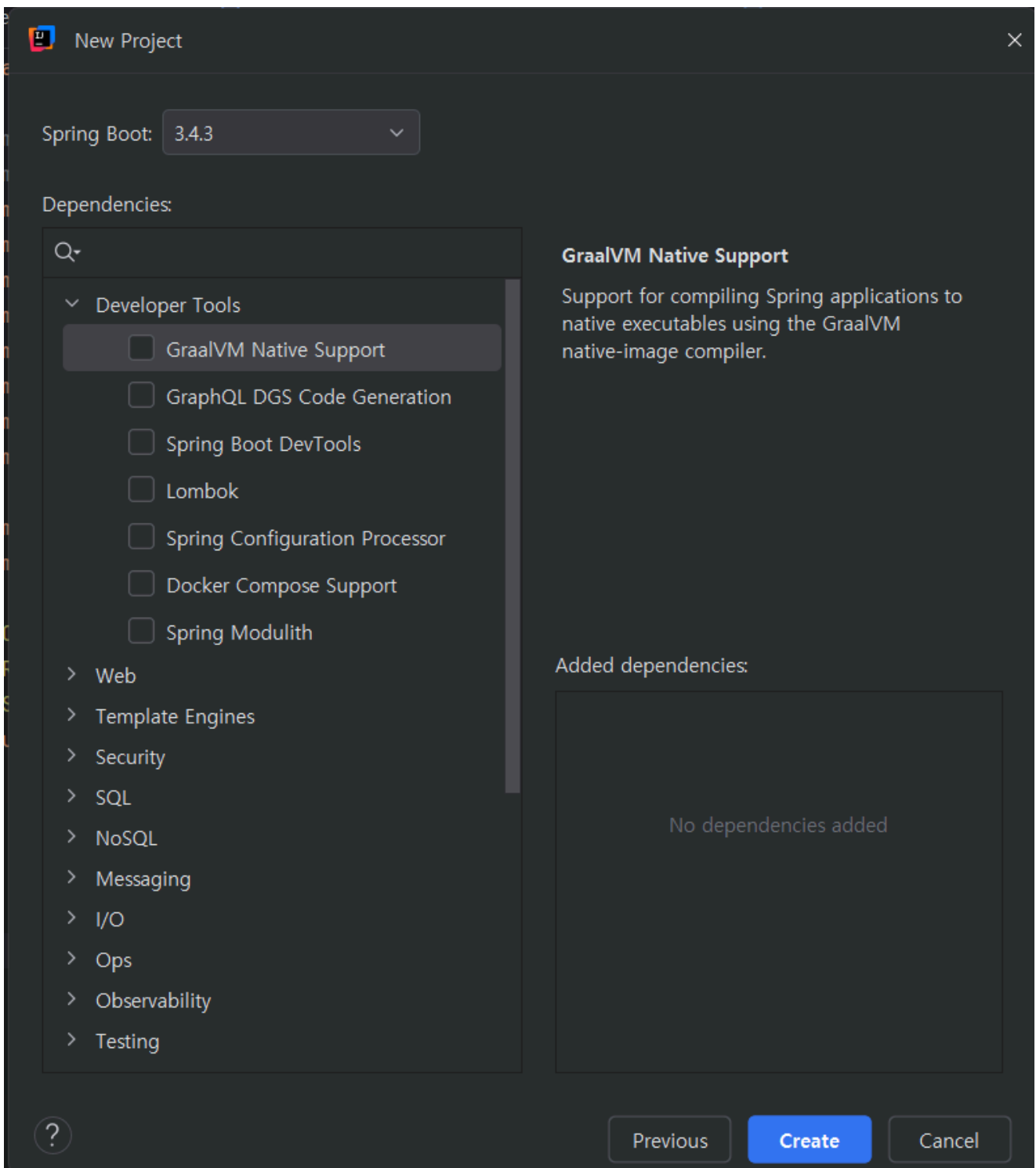
- 웹: start.spring.io
- CLI, curl, IDE(IntelliJ, STS, NetBeans 등)

✅ 프로젝트 시작 화면



항목	설명
Name	프로젝트 이름
Location	저장 위치
Language	Java, Kotlin, Groovy
Type	Gradle / Maven
Group	그룹 ID
Artifact	아티팩트 이름
Package name	패키지 경로
Java	JDK 버전

항목	설명
Packaging	JAR 권장 (독립 실행형)



의존성 주입 설정

- **Spring Web:** 웹 MVC, 톰캣, JSON 처리
- **DevTools:** 자동 재시작, 새로고침
- **Lombok:** 반복 코드 제거 (예: @Getter)
- **Thymeleaf:** HTML 통합 서버사이드 템플릿

프로젝트 파일 구조

폴더/파일	내용
src/main/java	소스 코드
src/test/java	테스트 코드
resources/	정적 리소스, 설정 파일
pom.xml	Maven 설정 파일
TacoCloudApplication.java	실행 클래스
application.properties	구성 설정
static/	이미지, CSS, JS 등 정적 파일
templates/	Thymeleaf 템플릿
TacoCloudApplicationTests.java	스프링 context 테스트

빌드 명세 확인

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
4      <modelVersion>4.0.0</modelVersion>
5      <parent>
6          <groupId>org.springframework.boot</groupId>
7          <artifactId>spring-boot-starter-parent</artifactId>
8          <version>3.4.3</version>
9          <relativePath/> <!-- lookup parent from repository -->
10     </parent>
11     <groupId>sia</groupId>
12     <artifactId>taco-cloud</artifactId>
13     <version>0.0.1-SNAPSHOT</version>
14     <name>taco-cloud</name>
15     <description>taco-cloud</description>
16     <url/>
17     <licenses>
18         <license/>
19     </licenses>
20     <developers>
21         <developer/>
22     </developers>
23     <scm>
24         <connection/>
25         <developerConnection/>
26         <tag/>
27         <url/>
28     </scm>
29     <properties>
30         <java.version>17</java.version>
31         <h2.version>1.4.19</h2.version>
32     </properties>
33     <dependencies> Edit Starters...
```

- <parent> : spring-boot-starter-parent 지정
- <dependencies> : 의존성 선언

애플리케이션 부트스트랩

```
package tacos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TacoCloudApplication {

    public static void main(String[] args) { SpringApplication.run(TacoCloudApplication.class, args); }

}
```

실행 가능한 JAR 파일로 애플리케이션 실행

```
@SpringBootApplication
public class TacoCloudApplication {
    public static void main(String[] args) {
        SpringApplication.run(TacoCloudApplication.class, args);
    }
}
```

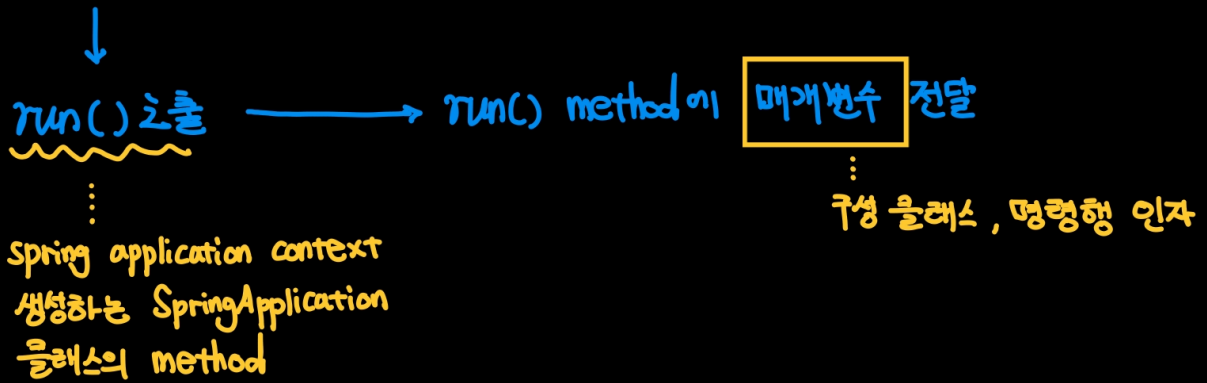
@SpringBootConfiguration ← 결합

- ① **SpringBootConfiguration**
: 구성 클래스로 지정
- ② **EnableAutoConfiguration**
: 스프링 부트 자동-구성 활성화
- ③ **ComponentScan**
: 컴포넌트 검색 활성화
ex) Controller, Service ...

main()

: JAR 파일 실행될 때 호출되어 실행되는 메소드.
: 대부분 표준화된 형태의 코드로 구성된다.

애플리케이션 시작



테스트 클래스

- 1개의 Test Method, 실행코드 X
- 성공적으로 코드될 수 있는 지 확인하는 기본적인 검사

@SpringBootTest : 스프링부트 기능으로 테스트 시작하라고 JUnit에 알려준다

1.3 스프링 애플리케이션 작성하기

홈 페이지를 구성해보자

💡 Controller 구성

```
@Controller
public class HomeController {
    @GetMapping("/")
    public String home() {
        return "home";
    }
}
```

- @Controller : 컴포넌트 등록
- @GetMapping("/") : 루트 요청 처리

@Controller : 컨트롤러

```
public class HomeController {
```

@GetMapping("/ ") : 루트 경로인 / 이 웹 요청처리

```
public String home() {
```

```
    return "home";
```

```
}
```

: View 이름 반환

```
}
```

💡 View 구성

- src/main/resources/templates/home.html 생성
- 공식 깃헙에서 예시 참조

🧪 Controller 테스트

```
@WebMvcTest(HomeController.class)
public class HomeControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    public void testHomePage() throws Exception {
        mockMvc.perform(get("/"))
            .andExpect(status().isOk())
            .andExpect(view().name("home"))
            .andExpect(content().string(containsString("원하는 문구")));
    }
}
```

- @WebMvcTest : Spring MVC 환경으로 테스트
- MockMvc : 요청 테스트 시뮬레이션

```

@WebMvcTest(HomeController.class)
public class HomeControllerTest {
    @Autowired
    private MockMvc mockMvc;

    @Test
    public void testHomePage() throws Exception {
        mockMvc.perform(get("/"))
            .andExpect(status().isOk())
            .andExpect(view().name("home"))
            .andExpect(content().string(
                containsString("Welcome to...")));
    }
}

```

▶ 빌드 및 실행

- localhost:8080/ 접속 시 home.html 렌더링
 - 톰캣 설치 없이 실행 가능 → **내장 서버 제공**
-

🔗 Spring DevTools

개발 중 생산성 향상

- 코드 변경 시 자동 재시작
 - 템플릿, 정적 리소스 변경 시 자동 새로고침
 - 템플릿 캐시 자동 비활성화
 - H2 DB 사용 시 자동 콘솔 활성화
-

🎯 Spring 목표

- Web, DB, 보안, Microservices 등의 반복 작업 최소화
- 의존성 관리 및 자동 구성

~~Spring in Action~~

(Spring → Spring application context, container 제공.)

↓
컴포넌트 생성/관리

Bean의 상호연결 → 'Dependency 주입'

① Configuration ⇒ 각 빈은 스프링 애플리케이션 컨텍스트에 제공하는 구성 클래스

```
public class ServiceConfiguration {
```

② Bean ⇒ 구성 클래스의 method.

```
public InventoryService inventoryService() {
```

```
    return new InventoryService();
```

```
}
```

③ Bean

```
public ProductService productService() {
```

```
    return new ProductService(inventoryService());
```

```
}
```

1) 스프링 프로젝트 설정.

- Name : 이름
- Location : 프로젝트 생성 경로
- Language : Java, Kotlin, Groovy
- Type : gradle / Maven
- Group : 프로젝트 그룹
- Artifact : 프로젝트 식별자
- packagename : 패키지 이름 (클래스나 파일이 속하는 패키지) ... 소스코드 상단에 선언
- JDK : JDK 버전
- Java : Java 버전은 설정한 JDK와 동일한 버전으로 선택
- Packaging : Spring boot는 war패키지 버전
Jar2 설정하면 application 독립 실행 o.
배포 / 유지보수 간소화